

Formal Verification of Lamport’s Mutex Algorithm for N processes

Max Moir
University of Canterbury
Christchurch, New Zealand
maxwmoir@gmail.com

March 16, 2026

1 Introduction

The mutual exclusion (mutex) problem, as originally introduced and solved by Dijkstra in [1], is a fundamental result in concurrent computing. Lamport’s Bakery Algorithm (LBA) is a solution to the mutex problem published in [2]. Lamport elaborates on it in [3], analysing several solution variants.

For $N \leq 5$ processes, LBA can be verified using a model checker such as TLA+, but exhaustive state expansion is not tractable for higher N . The goal of this project is to develop a formal model of LBA generalised to any finite number of concurrent processes. The model will be implemented in Lean4 to ensure that it maintains several required properties.

1.1 Problem Statement

The mutex problem is summarized by Dijkstra in [1] as follows.

To begin, consider N computers, each engaged in a process which, for our aims, can be regarded as cyclic. In each of the cycles a so-called “critical section” occurs and the computers have to be programmed in such a way that at any moment only one of these N cyclic processes is in its critical section.

A more detailed formalization of the problem was introduced by Lamport in [3], from which this proposal will take its terminology. As explained here, there are multiple properties that must always hold to prove the algorithm. This project will focus on the following two properties:

Definition 1.1 (Mutual Exclusion Property). For any pair of distinct processes i and j , no pair of operation executions $CS_i^{[k]}$ and $CS_j^{[k']}$ are concurrent.

Definition 1.2 (Deadlock Freedom Property). If there exists a nonterminating trying operation execution, then there exist an infinite number of critical section operation executions.

For a proof to be valid, these two properties must be shown to hold for every process in all state combinations. There are several variants of LBA that satisfy stronger requirements, but these are outside the scope of this project.

1.2 Formal verification in Lean4

Lean4 is a functional programming language and theorem prover with extensive mathematical tooling. Recently, it has been used to formalize and prove many problems in mathematics, including, for example, some of the Erdős problems. Currently, there are efforts in the Lean4 community to verify further problems, such as Andrew Wiles’ proof of Fermat’s Last Theorem. It can be used equally to ensure invariants within programs, and to prove mathematical theorems. This project will use Lean4 to implement an invariance proof that LBA ensures mutual exclusion and deadlock freedom.

2 Motivation

Formal verification is a topical field in modern Computer Science. Proving the correctness of programs is not only elegant in a theoretical sense, but is often necessary in mission-critical applications. Due to the difficulty in formalizing programs, reliability in industry is often left to automated testing. However, in specific circumstances where correctness must be ensured, formal proofs may be required.

This specific problem was chosen because it is a fundamental result in concurrent programming. Writing a formal proof for it acts as benchmark that displays the capabilities of formal verification. Additionally, it is challenging to verify the correctness of LBA for N processes without employing an inductive approach such as this one.

Lean4 was chosen for similar reasons. Recently, it has been receiving attention for the depth of its mathematical coverage. Its library `mathlib` provides mathematical foundations and data structures that will be useful in the proof of this algorithm. The decision in choosing Lean4 was also motivated by its contemporary relevancy.

3 Related Works

In addition to the problem statements by Lamport and Dijkstra mentioned previously, there are several other papers that will likely be included in the literature survey. For example, in [5], Nigro proves mutual exclusion for $N \leq 5$ processes using exhaustive model checking. This result is effective for limited processes but cannot show LBA holds for larger finite numbers of processes, due to the state space expansion problem. Earlier this year, Nigro and Cicirelli published another formalization ([6]) of LBA which uses a statistical model checking approach. This approach can scale for $N > 5$ processes, but is not as general as employing an inductive approach.

Another paper, [7], proves LBA satisfies the mutual exclusion protocol in CafeOBJ. [7] uses an approach similar to one that this project will use, but its implementation is more limited. It verifies that mutual exclusion holds, but does not verify other properties such as deadlock freedom, which this project aims to accomplish. The overall system may be modelled with a node for each process, as explained by Lamport in [4].

4 Methodology

This project will use a distributed state machine with invariance proofs that ensure mutual exclusion and deadlock freedom. These properties can be maintained by creating an inductive invariant, which is proved in each state transition. The overall system may be modelled with a node for each process, as explained by Lamport in [4].

This project will implement the one-bit variant of LBA, as defined in section 3.2 of [3]. It is also noted in [8] that the ticket numbers of waiting processes in LBA have the potential to grow indeterminately large. The paper poses a solution to this issue by moving ticket numbers around a ring of length $2N$. This project will follow the same approach.

The model will consist of methods used to transition processes between states, with a validity invariant ensured after each change. It will show that for any finite N independent processes, the mutual exclusion and deadlock freedom properties will always hold across the model's processes.

References

- [1] E. W. Dijkstra. Solution of a problem in concurrent programming control. *Communications of the ACM*, 8(9):569, Sept. 1965.
- [2] L. Lamport. A new solution of Dijkstra's concurrent programming problem. *Commun. ACM*, 17(8):453–455, Aug. 1974.
- [3] L. Lamport. The mutual exclusion problem: partII—statement and solutions. *Journal of the ACM*, 33(2):327–348, Apr. 1986.

- [4] L. Lamport. Deconstructing the bakery to build a distributed state machine. *Communications of the ACM*, 65(9):58–66, Sept. 2022.
- [5] L. Nigro and F. Cicirelli. Correctness Verification of Mutual Exclusion Algorithms by Model Checking. *Modelling*, 5(3):694–719, Sept. 2024.
- [6] L. Nigro and F. Cicirelli. Statistical Model Checking of Lamport’s Bakery Mutual Exclusion Algorithm. In N. Atulya K., J. Dharm Singh, M. Durgesh Kumar, and A. Joshi, editors, *Intelligent Sustainable Systems*, pages 142–152, Cham, 2026. Springer Nature Switzerland.
- [7] K. Ogata and K. Futatsugi. Formal Analysis of the Bakery Protocol with Consideration of Nonatomic Reads and Writes. In S. Liu, T. Maibaum, and K. Araki, editors, *Formal Methods and Software Engineering*, pages 187–206, Berlin, Heidelberg, 2008. Springer.
- [8] T.-K. Woo. A note on Lamport’s mutual exclusion algorithm. *ACM SIGOPS Operating Systems Review*, 24(4):78–80, Sept. 1990.